

ARGUS

Full Integration Audit — Modules 03-25

Checkpoint	Phase 1 — "Engineering Complete", before any real data
Scope	The composition of 25 individually-approved modules, not any module's internals
Branch	claude/new-session-96aymh
Commits	5c5383e (register) · 0904705 · 8bf9808 (startup logging) · 47177b8 (argv refusal)
Deliverable	docs/architecture/KNOWN_ISSUES.md — 19 entries, first consolidated register
Test suite	2,259 passing · 0 failed · 0 skipped · lint clean

Verdict

The composition holds. All six cross-module seams are intact, no module built after another quietly broke it, and the full suite passes in one process with no ordering dependency. Twenty-five individually-correct modules did add up to a coherent system — which is not automatically true and was the thing this audit existed to check.

The drift found is **in the enforcement mechanisms, not the code they enforce**. Two structural tests that exist to keep guarantees holding do not cover the way those guarantees would most likely be broken. Neither is a bug today; both are the reason a bug would not be caught tomorrow. That is precisely the class of thing 25 narrow sequential reviews cannot see and a composition audit can.

Separately, taking the system to production surfaced three real defects — two in the deployment layer's own robustness, one a latent database defect present since Module 11. All three are fixed or registered.

Part A — Test suite integrity

The measured total

```
2,259 passed · 0 failed · 0 skipped
```

One process, all modules, clean environment: no `.env` on disk, no `ARGUS_*` in the shell. This is the measured figure, not a sum of previously-reported per-module numbers.

Cross-module isolation: none found

The suite was re-run with the collection order reversed to expose ordering dependencies. One failure — a timing-sensitive performance assertion (20.4ms vs 21.7ms per security) that passes 8/8 twice when run alone. A flake, not order-dependence. No shared-fixture leakage, no migration-ordering problem, no global-state bleed between modules.

A methodological correction, reported on myself

The first reversed-order run reported three failures in `tests/unit/config` and `tests/unit/db`. Those were my own contamination — a `.env` file planted for the test below, while that background run was executing. I recognised the signature, re-ran clean, and they disappeared. Recorded because three unexplained failures left unchanged would have been a false finding in an audit whose whole job is to not produce those.

The `.env` isolation gap — still open, now proven

`tests/unit/config/conftest.py` clears every `ARGUS_*` key from the process environment. It cannot clear a `.env` file, because `AppConfig` declares `env_file=".env"` and `pydantic-settings` reads it from disk, never through `os.environ`.

```
# with a two-line .env planted:
FAILED tests/unit/config/test_environment.py::test_defaults_to_development
FAILED tests/unit/config/test_settings.py::test_partially_set_database_group_names_each_missing_fi
eld
2 failed, 31 passed

# the leak, visible inside the assertion itself:
input_value={'name': 'leaked_from_d...nv', 'host': 'myhost'}
```

Never fires in CI because `.env` is gitignored and never created. Fires on any developer machine that has run the app locally. Registered as B1, MEDIUM. Not fixed — pre-existing and already-flagged, which the audit's boundary puts out of scope. The fix is one line.

Part B — Cross-module seam integrity

#	Seam	Verdict
1	current_user_id (M19 seam / M22 implementation)	PASS
2	services/shared/ extracted conventions	PASS
3	Module 18 read surface	PASS
4	M17 approved_runs() / M20 release gate	PASS — enforcement gap
5	M23 ordering registry + probe()	PASS — coverage gap
6	Project-wide log scrubbing	PASS

1 — The identity seam

Exactly one commit touched Modules 19-21 after Module 22, and its complete diff is an additive optional `security=None` parameter plus a `harden(app)` wrap. **No route signature, no query, no schema changed.** No second identity mechanism exists. One name collision registered as F1: `intelligence/detail.py:resolve_identity` resolves a *security's* ticker, not a user.

2 — Shared conventions

One definition each of `error_payload`, `ApiError`, `Unavailable`, `Freshness`, `Provenance` — all in `services/shared/`, every consumer importing from there. Module 21's `Provenance` subclassing fix holds: the base carries only note, with exactly two per-service subclasses.

3 — Module 18's read surface

Every consumer outside `core/live_scanner/` goes through the published surface — `runs_by_status`, `scan_results`, `run_daily`. Nothing queries `live_scan_runs` directly.

4 — The publication gates: guarantee holds, net has a hole

Verified independently: `approved_runs()` has exactly one caller, and `gate.py` reaches results only through Module 17's loader. But the structural test scans for result tables named as *string literals* — the raw-SQL idiom, which ARGUS does not use. A planted bypass using the idiom it does use passed all 32 structural tests:

```
# planted into services/public_stats/aggregates.py
from infra.db.schema.setups import setup_outcomes # then: select(setup_outcomes)

32 passed in 1.84s          <-- NOT CAUGHT
```

5 — Ordering registry: complete, but scanned incompletely

All four hazards still registered `safe=False`, still unfixed. **No fifth instance exists** — verified by enumerating every `limit(1)` project-wide and re-scanning all 41 source files changed since Module 23. But the completeness test scans `core/`, `services/` and `data/` — not `infra/`, where Modules 24 and 25 put all of their code. I ran the same AST logic there: clean.

6 — Log scrubbing

Break attempt: a credential-shaped log field planted in `core/universe/` — a package that is neither identity nor observability, i.e. exactly what Module 24's project-wide extension was for. **Caught**, with file and line.

Part C — Consolidated known-issues register

Nineteen entries compiled into docs/architecture/KNOWN_ISSUES.md, each re-verified against the current tree rather than trusted from the report that filed it. **Three appear in no module report at all.**

The Module 16 mystery — resolved, and it is two functions

Module 25's report was cut off mid-sentence and named one occurrence. There are two. data_snapshot has a **unique** version_label and a **non-unique** content_checksum, and both publishers compute their idempotence key and their uniqueness key at different granularities:

```
checksum      = sha256(f"{config.content_checksum()}|{as_of.isoformat()}")    # microseconds
version_label = f"{config.version_label()}@{as_of.date().isoformat()}"      # days
```

Two calls, same config, same date, different instant: the checksums differ, so the "already published?" lookup misses; it inserts; the labels are identical; UniqueViolation. **The failure is a loud crash, not a silent overwrite** — the question the cut-off note left open. Confirmed empirically against a real migrated database:

```
Module 16  publish_outcome_snapshot      core/outcome_tracking/config.py:219
  same instant, twice      : IDEMPOTENT (same id returned)
  two instants, same day : UniqueViolation: uq_data_snapshot_version_label

Module 11  publish_similarity_configuration core/historical_similarity/config.py:196
  same instant, twice      : IDEMPOTENT (same id returned)
  two instants, same day : UniqueViolation: ... same constraint
```

Module 16's copy is worked around at its only live caller. **Module 11's has no workaround and appears in no module report.**

The register

ID	Item	Sev
A1	Four "latest row wins" ordering hazards — open by instruction; no fifth exists	HIGH
A2	data_snapshot publishers collide on two same-day calls — two functions	HIGH
A3	Gate structural test scans an idiom the codebase does not use (<i>audit finding</i>)	MEDIUM
A4	Ordering-registry completeness test does not scan infra/ (<i>audit finding</i>)	MEDIUM
A5	TOTP secrets stored as plaintext; no encryption mechanism exists (<i>audit finding</i>)	MEDIUM
B1	A local .env leaks past the suite's env isolation	MEDIUM
C1	No off-platform backup storage — largest gap	HIGH
C2	Dockerfile "never built" — corrected: it builds and runs	CLOSED
C3	Production health past startup unconfirmed	HIGH
C4	Rate limiter single-process — contained, not solved	MEDIUM
C5	No PITR — RPO is 24 hours	MEDIUM
C6	No load test	LOW
C7	Scanner never run in production shape	MEDIUM
C8	Append-only tables grow unbounded — measured, not prunable	LOW
D1	Account recovery deferred; interacts with hard MFA requirement for admin	MEDIUM

D2	Cookie sessions deferred — reasoning unchanged, public page is unauthenticated	LOW
E1	fundamentals.py filter drift — RESOLVED	CLOSED
E2	TLS / HSTS — RESOLVED by Module 25	CLOSED
E3	Log retention — RESOLVED by Module 25	CLOSED

A3 and A4 are the same shape twice, and worth reading together: in both cases the guarantee holds, and the test that exists to keep it holding does not cover the way it would most likely be broken.

Part D — Production deployment

Reported as blocked in the first pass — the audit environment's egress proxy refused CONNECT to every Railway host, and no workaround was attempted. Bringing the remaining six services up then surfaced three real defects, two of them in Module 25's own deployment layer.

Defect 1 — Entrypoints silently discarded their arguments

Fixed — commit 47177b8. This was the live incident.

A service was configured with `python -m infra.deploy.migrate && uvicorn ...`. Reaching a shell, `&&` sequences two commands. Executed without one, `&&` and everything after it become `argv` for the migration — which discarded them with `_ = argv`, migrated nothing, printed "schema already at head", and exited 0.

```
$ python -m infra.deploy.migrate '&&' uvicorn infra.deploy.asgi:identity_app --factory ...
EXIT CODE = 0                                <-- exits 0, "successfully"
last log line: "schema already at head"
uvicorn startup lines: 0
listeners on 8942:      0
```

Every observable was consistent with success: the platform saw a process that ran and ended, no server started, the health check failed five minutes later, the domain returned 502. The one thing that would have named it instantly was sitting in `argv`, thrown away.

`infra/deploy/cli.py` now refuses: it names the arguments verbatim, exits 2, and when the list begins with a shell operator says so explicitly and prints the `sh -c '...'` form that guarantees a shell. Refusal happens before logging is configured and before any database work.

Defect 2 — A failing service factory could die without saying anything

Fixed — commit 8bf9808.

Under `--workers`, a factory exception is raised in a child process whose `stderr` a hosted platform may or may not surface. From the outside, "the factory raised" and "the factory was never called" were indistinguishable. `_module_app` now announces entry on `stderr` before anything can fail — its *absence* is the diagnostic — and reports any failure on two independent channels before re-raising, because the structured channel depends on logging having worked, and logging is one of the things that can break.

Defect 3 — TOTP secrets are stored in plaintext

Registered as A5, not yet fixed. Found while checking a claim about MFA configuration.

```
# services/identity/mfa.py
secret = pyotp.random_base32()
.values(mfa_secret=secret, ...)          # stored raw

# infra/db/schema/users.py
Column("mfa_secret", Text, nullable=True) # plain Text, no encryption

$ grep -rn "Fernet|AES|encrypt\" core/ services/ infra/ data/ packages/
NONE - no encryption-at-rest mechanism exists
```

Anyone with a database dump — including any `pg_dump` the backup tooling produces — can mint valid TOTP codes for every enrolled admin. Scrubbing keeps the secret out of logs; nothing protects it at rest. MEDIUM today with one admin and no real users; HIGH the moment either changes, or the backup archive lands off-platform.

What remains unverified

Health past startup for the six new services, TLS termination behaviour, the platform's own health-check polling, the pre-deploy hook in place, and the cron schedules firing. C3 stays open until each is observed rather than

inferred.

Assessment

Is the system what 25 approved reports implied?

Substantially yes, with one instructive exception. Every seam held. Two items expected to be open were already closed — `fundamentals.py`'s filter drift, and TLS and log retention, both delivered by Module 25 — which is the process working as designed.

Where it drifted

The enforcement mechanisms, not the code they enforce. A3 scans for an idiom the codebase does not use; A4 scans every package except the two where the newest code lives. In both cases the guarantee is intact and the net has a hole shaped like the likeliest mistake. That is the class of finding a composition audit exists for.

The second real finding is A2's second copy. Module 25 found a defect, described half of it, and was cut off. The half it never reached was that the same bug sits unmitigated in Module 11, written five modules earlier. A register like the one this audit produced would have caught that when Module 16 was written — which is the argument for the register existing at all.

Ranked, before real data lands

#	Item	Why it is first
1	A2 — Module 11's <code>publish_similarity_configuration</code>	Unhandled <code>IntegrityError</code> on a path that believes it is idempotent, with no workaround. Small, contained fix.
2	C1 — off-platform backup storage	The restore drill works and passes; the destination does not exist. DR rests entirely on provider snapshots nobody has verified.
3	C3 — production health past startup	Nothing about the running system is confirmed beyond the fact that it starts.
4	A5 — MFA secrets at rest	Cheap to fix now; expensive to discover later. Rises to HIGH with real users.

A note on this audit's own limits

Part D was blocked, then partially completed, and is still not fully verified — it is unperformed in places, not passed. I also found my own contamination in a background test run mid-audit, caught it, and re-ran. That is worth stating because it is the same lesson the deployment taught twice over: **a green suite is a claim about the conditions it ran under**. Two thousand tests passed while a deployed process could not start, and passed again while a migration silently swallowed the rest of its own start command. The defects fixed here are not that something failed — it is that failing looked identical to succeeding.